

Webpack & Vite

Integrantes:

Bitti Guido	48937
Cosimo Ivo	52618
Mora Joaquin	51388
Ulla Lucas	53246

De qué vamos a hablar hoy

01 Conceptos comunes

02 Vite en profundidad

03 Webpack en profundidad

04 Evaluación y Resultados

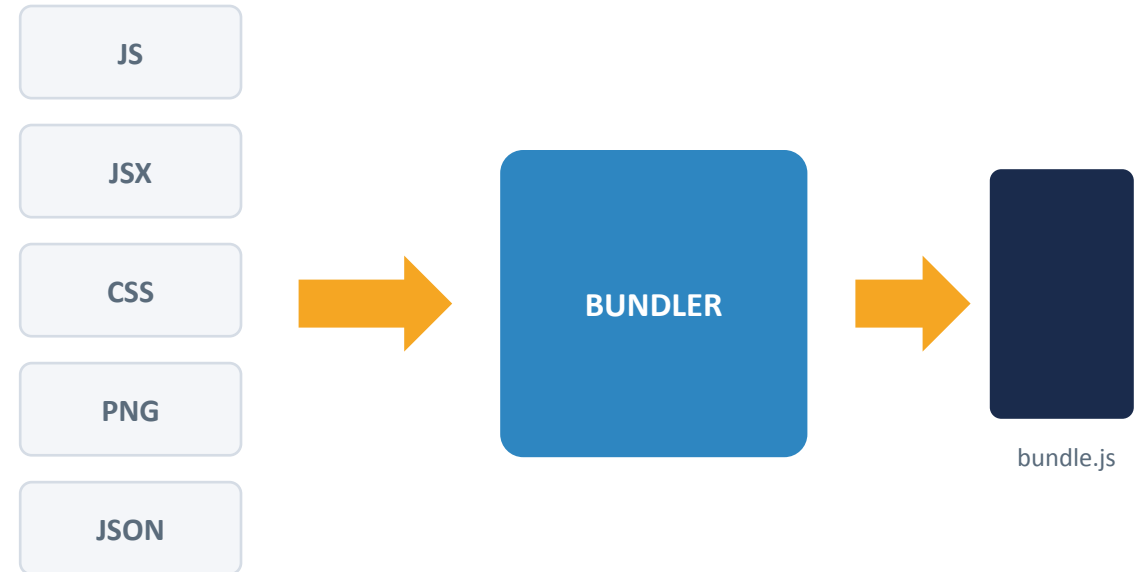
05 Demo en vivo

06 Conclusiones

¿Qué es un bundler?

Un bundler (empaquetador de módulos) toma un proyecto compuesto por muchos archivos -JavaScript, CSS, imágenes, fuentes- y los combina en uno o varios archivos finales estáticos, optimizados para que el navegador los consuma de forma eficiente.

Ambas tecnologías que comparamos en esta PoC -Webpack y Vite- son bundlers: resuelven el mismo problema de fondo, pero con arquitecturas distintas por dentro.



¿Por qué usar un bundler?

1

Rendimiento

Menos archivos y requests HTTP hacia el navegador, con archivos optimizados y comprimidos.

2

Compatibilidad

Transpilación y polyfills para que el código moderno funcione en navegadores más viejos.

3

Organización

Permite dividir el código en módulos pequeños y reutilizables sin penalizar al usuario final.

4

Optimización

Minificación, tree shaking y code splitting: solo se descarga el código que realmente se usa.

Principales Funcionalidades

1

Code Splitting

División del código en fragmentos (chunks) que se cargan bajo demanda, en vez de descargar toda la app de una vez.

2

Tree shaking

Elimina del bundle final el código que se importó pero nunca se usa, reduciendo el peso de lo que descarga el usuario.

3

Hot Module Replacement (HMR)

Reemplaza módulos en el navegador mientras programas, sin recargar la página ni perder el estado de la app.

4

Configuración por entorno

Permite tener comportamientos distintos en desarrollo (velocidad, debug) y en producción (optimización, minificación).



PARTE 2

Vite

Filosofía de Vite

Vite (palabra en francés para "rápido", pronunciado como /vi:t/, como "veet") es una herramienta de compilación que tiene como objetivo proporcionar una experiencia de desarrollo más rápida y ágil para proyectos web modernos.

Vite busca ofrecer una herramienta liviana, extensible y enfocada en el rendimiento.

Núcleo liviano

API reducida + sistema de plugins

Rendimiento

HMR rápido + Oxc + Rolldown

Web moderna

ESM y estándares modernos

¿Cómo funciona? — Dos Entornos Distintos

Vite mejora el tiempo de inicio del servidor de desarrollo dividiendo primero los módulos de una aplicación en dos categorías: dependencias y código fuente.

1

Desarrollo

- Servidor basado en ES Modules nativos.
- Carga los módulos bajo demanda.
- HMR extremadamente rápido: actualiza solo lo que cambió.

2

Producción (Build)

- Vite build utiliza Rolldown para empaquetar.
- Optimiza el código y genera recursos estáticos.
- Resultado listo para desplegar en producción.

Hot Module Replacement Nativo

Actualización instantánea

- Detecta qué módulo fue modificado.
- Actualiza únicamente ese módulo.
- No recarga toda la página.
- Mantiene el estado de la aplicación.

Integración con frameworks

- API de HMR basada en ESM nativo.
- Integración oficial con React, Vue y Preact.
- React Fast Refresh y Vue HMR.
- Viene preconfigurado al usar create-vite.

Configuración Cero o Mínima

Listo para usar

- Valores predeterminados para los casos más comunes.
- Permite crear un proyecto y comenzar rápidamente.
- Menos tiempo dedicado a configurar herramientas.

Configuración cuando hace falta

- No busca cubrir todos los casos de uso.
- Las necesidades específicas pueden resolverse mediante plugins.
- Se puede personalizar mediante vite.config.

Núcleo liviano

- Mantiene una API reducida y simple.
- Evita incorporar funcionalidades innecesarias al núcleo.
- Favorece un proyecto sostenible y extensible.

Características principales

1

**Servidor de Desarrollo
Ultrarrápido**

2

Soporte de ESM Nativos

3

**Compilaciones de Producción
Ultrarrápidas**

4

**Integración con Herramientas de
Desarrollo**

5

Ecosistema de plugins



PARTE 3

Webpack

Gobernanza actual

Technical Steering Committee (TSC)

Toma las decisiones técnicas y de dirección estratégica del proyecto.

Core Working Group

Grupo central que sostiene el desarrollo y mantenimiento día a día.

OpenJS Foundation

Ambos niveles operan dentro del paraguas institucional de la OpenJS Foundation, lo que le da a Webpack continuidad más allá de cualquier empresa o mantenedor individual.

Niveles de patrocinio (Support the Team) → Proyecto comunitario e independiente

Platinum

Gold

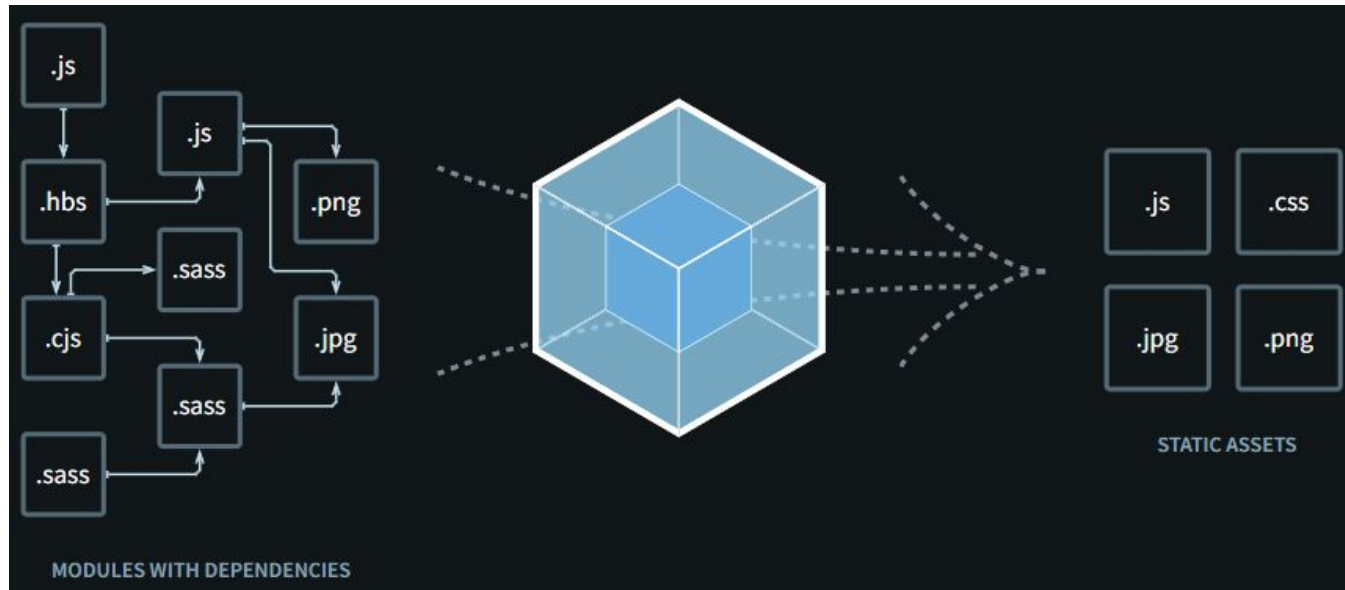
Silver

Bronze

Backers

<https://webpack.js.org/>

¿Cómo funciona? — El proceso de bundling



Entry & Output

Entry — punto de entrada

Le indica a Webpack desde qué archivo/s empezar a construir el grafo de dependencias.

```
// webpack.common.js

entry: './src/index.jsx',
```

Output — archivo(s) de salida

Define dónde se escriben los bundles finales generados.

```
// webpack.common.js

output: {
  path: path.resolve(__dirname, 'dist'),
  filename: '[name].[contenthash].js',
  clean: true,
}
```

Loaders, Plugins y Mode

Loaders

Transforman archivo por archivo (JSX, CSS, imágenes...) para que el bundler lo entienda.

 // webpack.common.js

```
module: {
  rules: [{
    test: /\.jsx?$/,
    use: 'babel-loader',
  ]},
}
```

Plugins

Actúan sobre todo el proceso de build, no archivo por archivo.

 // webpack.common.js

```
plugins: [
  new HtmlWebpackPlugin({
    template:
      './public/index.html',
  }),
]
```

Mode

Activa optimizaciones y comportamientos distintos según el entorno.



```
// webpack.dev.js
mode: 'development'
```

```
// webpack.prod.js
mode: 'production'
```

Características principales

1

Transpila todo antes de servir

Procesa la app completa (código propio + dependencias) antes de poder servir nada, tanto en desarrollo como en producción.

2

Loaders y ecosistema

Cualquier tipo de archivo se procesa mediante loaders encadenables, con un ecosistema dedicado (webpack-contrib) de cientos de loaders y plugins ya hechos.

3

Configuración explícita y granular

Cada paso del proceso (entry, loaders, plugins, output) se declara a mano, dando control total sobre el build.

4

Asset Modules

Manejo nativo de imágenes y fuentes desde Webpack 5, sin loaders externos.

5

Module Federation

Permite compartir código entre builds independientes en tiempo de ejecución — clave para micro-frontends.

Ventajas y desventajas



Ventajas

- Ecosistema de plugins más grande y maduro del mercado
- Máxima flexibilidad de configuración
- Estándar de facto en proyectos grandes, con años de estabilidad



Desventajas

- Configuración verbosa y con curva de aprendizaje pronunciada
- Dev server más lento en proyectos grandes (bundlea antes de servir)
- Menor atractivo para nuevos desarrolladores frente a alternativas modernas



PARTE FINAL

Comparación práctica

Metodología, resultados medidos sobre proyectos equivalentes y demo en vivo

Características evaluadas

Técnicas

- Tiempo de build de producción
- Tiempo de arranque del dev server
- Tamaño del bundle final
- Code splitting
- Hot Module Replacement

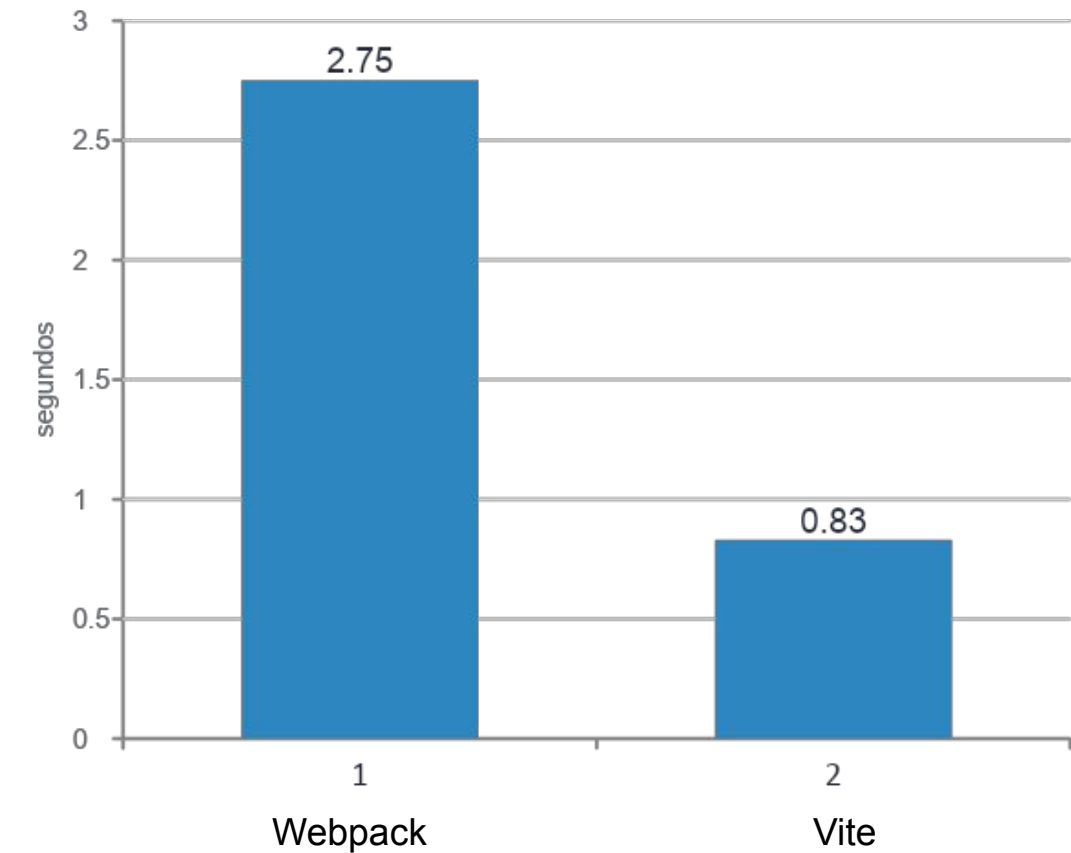
De proceso

- Complejidad de configuración (líneas/archivos)
- Cantidad de dependencias de build
- Opinión de la comunidad

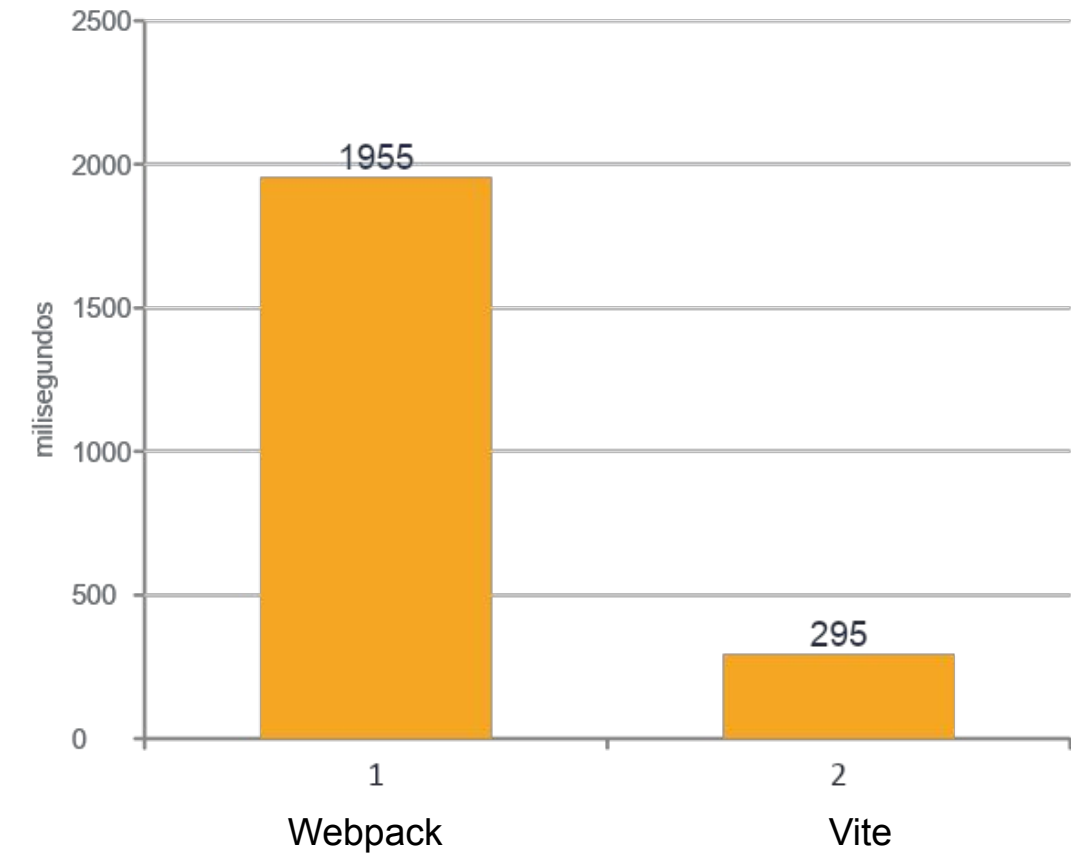
Las características técnicas se midieron sobre dos proyectos equivalentes: el mismo CRUD de tareas en React, con las mismas rutas y el mismo backend simulado (json-server), implementado una vez con Webpack y otra con Vite.

Velocidad: build y dev server

Build de producción (s) — corrida en caliente

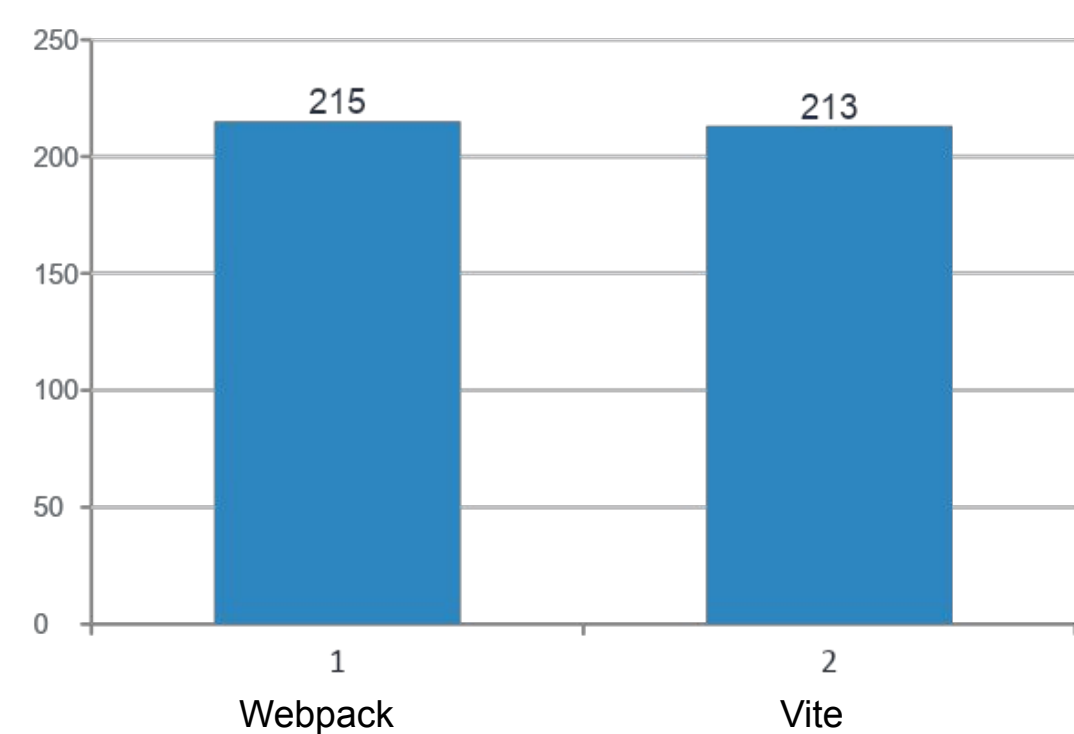


Arranque del dev server (ms) — promedio



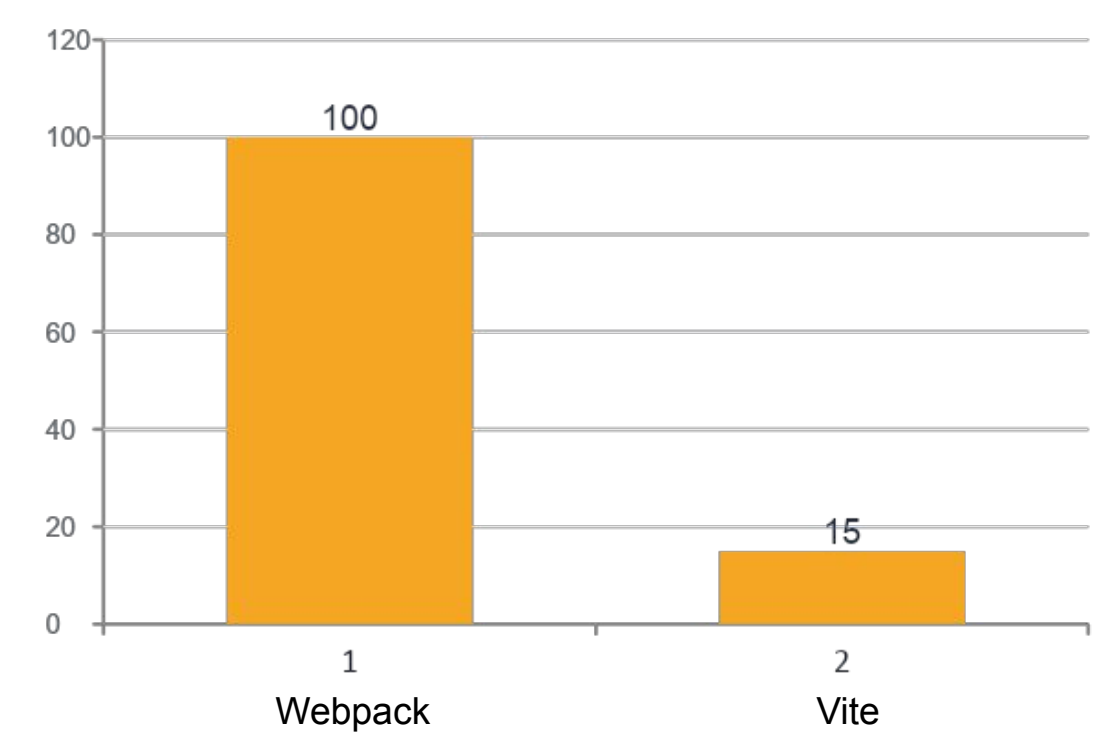
Tamaño de bundle y complejidad de config

Bundle final (KB)



Prácticamente idénticos: el tamaño final no es el diferenciador en este proyecto.

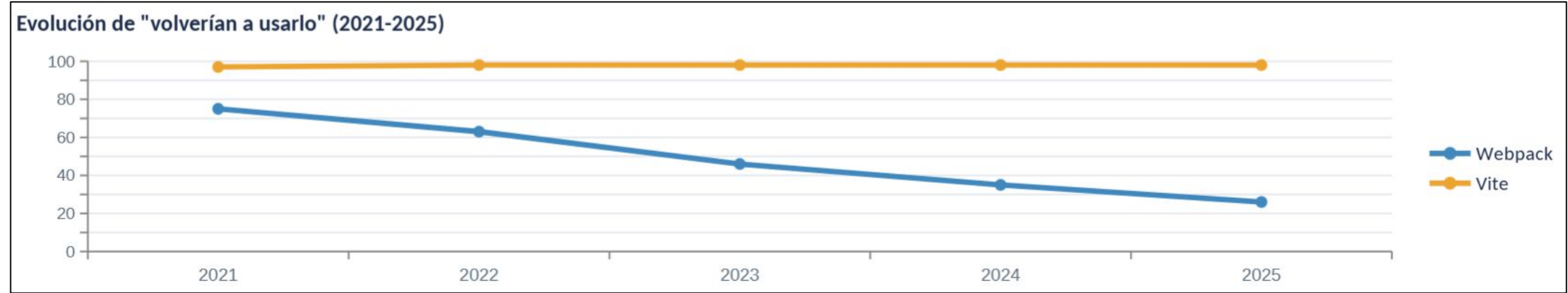
Líneas de configuración necesarias



Webpack: 4 archivos de config. Vite: 1 archivo. (15 dependencias de build vs 4)

Webpack vs Vite en la comunidad

Fuente: State of JavaScript 2025 — Build Tools (datos exportados, edición 2025)



Qué vamos a mostrar

01

CRUD completo

Crear, listar, editar y eliminar tareas contra la API local (json-server).

02

Code splitting

Pestaña Network del navegador: cada ruta descarga su propio chunk bajo demanda.

03

HMR en vivo

Editar un componente y ver el cambio reflejado sin recargar ni perder el estado.

04

Dev Server y Build de producción

Compilación del servidor local y run del dist.

CONCLUSIÓN

¿Cuándo conviene cada herramienta?

Elegí Webpack si...

Flexibilidad y ecosistema por sobre velocidad

- El proyecto ya existe y tiene una config Webpack madura
- Necesitás Module Federation / micro-frontends
- Requerís control muy fino sobre cada paso del build

Elegí Vite si...

Velocidad y simplicidad para proyectos nuevos

- Arrancás un proyecto nuevo desde cero
- Priorizás velocidad de desarrollo (dev server casi instantáneo)
- Preferís configuración mínima con buenos defaults